ETL Developer Guide

How to structure your ETL package so the platform can run it correctly

1. What is an ETL Package?

An ETL package is a `.zip` file you upload to the platform. It contains your Python scripts, a config file, and a requirements file. The platform extracts it, installs dependencies in an isolated virtual environment, patches the config with any user overrides, and runs your entry point script.

the platform never runs scripts as root and never modifies your original ZIP, every execution gets a clean copy.

2. Required Files

File Type	Required	Description
.py	YES	Entry point : the script the platform runs
.json / .Yaml	YES	All runtime parameters (paths, dates, settings)
.txt	Recommended	Python dependencies

Everything else (helper modules, SQL files, templates, mappings) is optional but must be inside the zip.

3. ZIP structure

Rules:

- Do NOT nest your files inside an extra folder inside the zip. The platform looks for the main file and configuration file at the root or one level deep.
- Do NOT include `.venv/`, `venv/`, `\_\_pycache\_\_/`, `.git/` — they are stripped automatically.
- Keep the zip under a reasonable size. Large input data files should NOT be inside the zip — they are referenced by path in the config

Example : my_etl.zip
├── main.py
├── config.json
└── requirements.txt

Configuration file Rules

The config file is the contract between your script and the platform. It is read, patched with user overrides at launch time, and passed to your script.

- any name is valid (.json / yaml)
- Every folder your script writes to must appear in the config
- All paths must come from config , never hardcode in the script
- Keys with spaces in their names can cause parsing issues
- Output folder paths must end with `/`
- Date fields use ISO format: `YYYY-MM-DD` or `YYYY-MM`
- Booleans must be real JSON booleans: `true`/`false`, not strings
- Every folder your script writes to must be in the config
- **Date field auto-detection** — keys containing any of these words get a date picker in the UI: `date`, `from`, `to`, `start`, `end`, `period`, `month`, `year`, `since`, `until`
- **Number field auto-detection** — keys containing any of these words get a number input: `count`, `number`, `nb`, `max`, `min`, `limit`, `size`, `threshold`, `timeout`, `port`
- **Folder Block** Use when an input folder must contain specific files the platform should verify before launch:

Folder Block (for input folders with file checks)

json

```
"input_folder": {
  "path": "C:/Data/Source/",
  "files number": 10,
  "check_mode": "both",
  "file1": "report.xlsx",
"file2": "parc*S*.xlsx",
"file3": "*.csv",
"file4": "-"
}
```

`check_mode` options:

- `count` only checks total file count matches `files number`
- `files` only checks each `fileN` entry exists
- `both` checks count AND each `fileN`
- `auto` (default) uses `files` if fileN entries exist, `count` otherwise

`fileN` pattern syntax:

- `report.xlsx` exact match, case-insensitive
- `report*` starts with "report"
- `*_final.xlsx` ends with "_final.xlsx"
- `*parc*S*` contains "parc" and "S" anywhere
- `*.csv` any file with that extension
- `.xlsx` shorthand for `*.xlsx`
- `-` — placeholder, not set yet; shows a warning but does not block launch

Script Rules

- Read config with `open("config.json")`
- Wrap logic in `main()`, call under `if __name__ == "__main__"`
- Exit code `0` = success, anything else = FAILED
- Write at least one output file , no output file = FAILED even on exit `0`
- Print progress to stdout, it shows in the UI

Path Classifications

During the launch flow, every path-like config key gets classified as:

- `input` platform checks it exists before running; syncs it into the work directory
- `output` platform creates the directory automatically and scans it for result files after the run
- `skip` ignored

Output Files Collected Extensions the platform registers after a run: `.xlsx`, `.xls`, `.csv`, `.pdf`, `.zip`, `.json`, `.txt`

requirements.txt

- Always include one, even if nearly empty
- Pin versions: `pandas==2.1.4`
- Changing it triggers a full venv reinstall on the next run